

FRED TALKS

26th September 2026

CONTENTS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 1776 WORDS

the senior engineer death spiral

SUNIL PAI · 959 WORDS

How not to forget what matters

HENRIK KARLSSON · 2660 WORDS

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 2024 WORDS

Some new agentic patterns

MASSIVELY PARALLEL PROCRASTINATION · 08 JUL 2026 · [SOURCE](#)

You can also read this post on [Prime Radiant's blog](#).

Almost all of this post was written on June 1, before Anthropic shipped Claude Tag. Nothing I'm writing about uses (or is based on) Claude Tag

At [Prime Radiant](#), we've got a bunch of agents in our Slack. I want to tell you a bit about how we've started shipping some changes to internal production with an "agentic user in the loop." Letting an agent work directly with the implementer building its runloop and tooling is getting us higher-quality tools and experiences for the agent without a human in the middle of a game of telephone.

But first, a bit of backstory on how we're using non-coding agents at Prime Radiant.

One of them, Scribble, is just there to listen for when someone reports a problem or provides a new bit of information. It opens tickets for the issues and updates an internal wiki with the information. (It also logs our daily standups and keeps us honest about whether we finished what we said we were going to do the previous day.)

Another, Nora, is our junior go-to-market "person." Nora is built on top of Nanoclaw and is still learning her craft, but is surprisingly helpful, partially because she considers herself a team member, not just an assistant. I've put a bunch of time into tuning her prompting and constitution. She gets a ton of value out of knowing that she journals obsessively. (I should note that I don't tend to gender my agents, but Nora picked a name and a gender and was very clear about that and...who am I to argue?)

I think the most surprising interaction I've had with Nora was the morning I woke up to a slack DM telling me that I needed to be speaking at more conferences. (I do.) She pointed out that the CFP for a particular AI conference had closed a week earlier, but that the program chair was known to take late submissions of interesting talks by DM. She'd taken the liberty of drafting a DM for me to send. I didn't actually *send* the DM, but I should have.

We have another agent, "Spec-together" that was a first prototype of what has become our new multi-player [Brainstorm](#) app.

And we have a small fleet of "Sen" personal assistant agents. They're **claw-esque*, but date from before openclaw was a thing. Each one has been trained to be an executive assistant, using a set of skills built on some of the resources that excellent human EAs use to learn their craft. Sen triages my mail, puts together a daily brief, does research tasks (both for me and independently), interacts with Linear, etc.

They have the standard set of affordances - They can chat on Slack. They use tools. They've got a task scheduler that can wake them up and re-inject a prompt they set for themselves at a specific time or on a schedule they define. They can use and author skills. Mine has a dropbox folder it shares with me. When we moved Sen to the cloud, I lost one of my favorite dumb features - Sen used to be able to print things directly. Sen (v1) was built on top of the Claude Agents SDK.

For the past month or so, I've been doing initial bringup of a "new" iteration of Sen, intended to be more of a colleague than an assistant. This time around, I'm building it from the ground up on top of an agents SDK I control. Lace got its start in May 2025 as my take on a command line coding agent. Over the past year, it's mutated a few times, first growing a web interface, then flipping to an ACP-derived wire protocol so you can connect any client you want to it. It speaks a bunch of different flavors of model provider API, manages caching, subagents, tools, skills, and all the other stuff you might expect. It has supported subagents in isolated containers for the better part of a year. One of the latest things to land has been the ability to run subagents locally and to project *all* of their tools into a container, such that the agent believes it is running in that container.

The thing I've been working on over the past few weeks is the credentials story for the new Sen agent running on top of Lace. Right now, I'm building with a design that assumes the agent has its own credentials for any service that it accesses, not yours. (You wouldn't share your email account or GitHub credentials with a colleague, right?) As of now, nobody has solved Simon's Lethal Trifecta - If a single agent has access to private information, can communicate externally, and is exposed to untrusted content, there is no structural way to guarantee that that agent can't be suborned. So the name of the game is compartmentalization and risk *reduction*.

The architecture I've landed on for now:

- The main agent does not have the ability to directly communicate externally.
- The main agent is able to communicate with ephemeral subagents that *do* have the ability to communicate externally

- No agent has intentional direct exposure to credentials (with one very real gap that I haven't solved yet)

All credentials for the agent live in a 1Password vault. All outgoing https traffic goes through a onecli-style transparent MITM (man in the middle) proxy. When a subagent wants to do something that requires a credential, it runs a command that asks an *arbiter* agent running in another container for permission to use that credential. If the arbiter decides the request is reasonable, it provides the subagent with a random string they can use in place of the requested credential. When the agent uses that random string in an outbound request to the right remote host, the MITM proxy rewrites the random string into the real credential on the fly.

The case that's a little more complicated is when the agent wants to do something that requires a browser, like logging into Gmail as themselves or using a service that has no API. For a bunch of reasons, the MITM rewrite trick does not work as reliably in a browser context. Often the site/app's javascript plays games with the password to hash the password before sending it. Or does some other weird validation. Or is actually sending it to a different backend service. So far, the best I've come up with keeps the credential out of the agent's transcript and prevents unintentional disclosure. In these cases, the subagent running in the container generates a random temporary password, uses [superpowers-chrome](#) to fill it into the password field, and then runs a command that asks the arbiter agent (running in its own container) for help. The arbiter, if it decides the request is reasonable, instructs a helper tool to log into the subagent's container and replace the random password with the real one. It's not perfect and there's more we can do to lock things down, but it's a better first pass than anything I've had to date.

What actually got me writing today was not this architecture, but the *development pattern* I've been using to build this. On the one side, I've got a Claude Code session. I hooked it up to our Slack instance with a self-provisioned commandline slack client Drew built called [Slackline](#) - Slackline is designed to make it easy for agents that don't *live* on Slack to participate in discussions there. It has primitives for sending messages, reading chats, waiting for replies, etc. The whole command-line slackline tool has been built for agents as its primary user. Sure, a human *could* use it to use slack from the commandline. And sure, it has enough of a json api that you could use it as part of traditional automation. But it's *for* agents.

It took a bit of iteration to get to the point where the new Sen 2.0 agentic colleague harness was live on Slack. But once it was there, I could talk through its experience of its tools and memory and credentials and workspaces. And the issues showed up pretty quickly.

The "right" kind of context compaction for a coding agent is desperately wrong for a long-lived persona that might be engaging in multiple simultaneous conversations. And the only way to see if the credentials management tools work is to try to do something crazy, like sign into GitHub from a subagent session.

And sure, I could take this feedback into a Claude Code (or Codex) session and work it through. But I've got too many projects in flight and there's very little reason for me to be in the loop on most of this work. I would just slow it down. So I told Claude Code to use slackline to talk to "@Ada-sen" about the work it was doing.

Claude pinged Ada in #bot-testing to describe the work we'd just done and to ask Ada if it could test it out. Ada dutifully fired up a subagent and tried to log into GitHub. Claude watched the logs to see if the credential proxy did the right thing. It did not.

That kicked off a number of days of iteration. Claude would propose a change to Ada. Ada would review Claude's spec, raising questions or concerns. Claude would update the spec. Once Claude and Ada were in agreement, Claude would build an updated version of Ada, make sure Ada wasn't in the middle of something, and then deploy the update. Once Ada was up and running again, Claude would @Ada-sen, explain the test it wanted to run, and wait for results.

Once a bit of functionality works, Claude checks in with Ada to see what could make it simpler and easier to use. Ada typically kicks off some subagents to see whether they stumble through the new feature and then reports back.

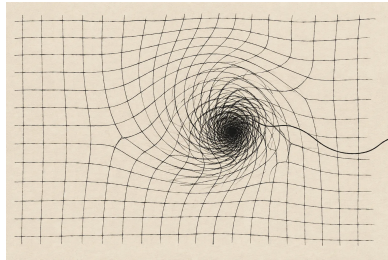
At one point, I wrote to Claude "I've got to get to bed. When this project is done, check with Ada to see whatever quality of life features you could build." I woke up to a laundry list of about a dozen harness improvements. Overnight, Claude had solicited a wishlist from Ada. Ada ranked a bunch of requests. Claude ordered them and wrote out a spec for everything it was comfortable building. Ada reviewed the specs, then Claude built the features. Ada tested them out and requested changes. Claude updated the features until Ada was happy. Once Ada signed off, Claude merged the changes to main and wrote up an after-action report for me.

It's been pretty magical to watch.

the senior engineer death spiral

SUNIL PAI · 19 SEP 2026 · [SOURCE](#)

(a friend recently got a new job, a very senior role, very well paid, and pretty different from his previous gig. he was asking me how to do a 60 to 80-hour week, how to do well enough to get promoted, etc. and as we talked, I realised he was feeling a bit of imposter syndrome and really wanted to prove himself. I ended up giving him a bit of a monologue. this is more or less what I told him.)



A hand-drawn grid spiraling into a dense knot, with one loose line escaping to the edge.

okay, so this is the most common failure mode I've seen. I call it the senior engineer death spiral. it usually happens when an engineer goes into a new job, gets a promotion, or even just gets a big project at work. or they ask for a big project because they think to themselves, "oh, if I work harder, do a bigger thing, then I will be rewarded with promotions and what have you," right? like, that's the move. and the thing they do is they tell themselves they need to almost cosplay being a more senior engineer than they are. they're like, "oh, you know what, I'm going to try designing something way more ambitious."

and what happens is they start disappearing for longer periods of time. like, for two, three weeks, you won't hear anything from them. occasionally, during the standup, they'll give what I call the positive update. "yeah, things are going well, you guys. I'll have something to show you quite soon. if you have any questions, reach out." and no one usually reaches out, but they won't really have anything to show.

and in the back of their head, right, they've started the spiral already. they're telling themselves, "oh my god, I haven't really shipped anything in a while. what I need to do is work harder. I'll just work hard enough. I'll save this and no one will know." like, "in the next week, I'll do a whole month's work. and no one will know." they'll start losing sleep, all of that. and it sucks, because bro, they start getting depressed, missing meals. their relationships suffer, both work and personal. and they just start spiraling, spiraling, and it ends up in worst-case scenarios. they burn out, they need to take a month off, they get PIP'd, they get fired, they quit their jobs because they think, "you know what, this is not salvageable." and it really sucks.

and by the way, this isn't something that I've noticed only with other people. this has happened to me. and it's happened so many times that I now recognize it early enough to take a step back and fix it.

see, man, the bigger point here is that because of COVID and coding agents and remote work, things that have happened in the last five to ten years, right, you don't really have the same structure of working next to someone and being given Jira tasks, etc. you have a lot more ownership, and you are unfortunately a little more siloed. so you never want to be in a situation where people don't know what's up with you. like, they shouldn't be saying, "yeah, I don't know what Sunil's been doing," and starting to get concerned, because they might only tell you when it's too late. you want to be in a position where "Sunil doesn't shut the fuck up about what he's working on," right? again, you don't have to be obnoxious about it. you have to share your work.

I think in moments like this, the thing you need to do is actually a little counterintuitive. but you have to start from a position of assuming that everyone around you is operating in good faith, okay? like, they hired you for you. they didn't hire you for who you think you're going to be in six months. they like you for who you are right now, bro.

so the thing to do is actually to drop a level. not to try to be a higher-level engineer, but to almost drop a level and simply become the greatest teammate for a while. you're still responsible for your work, you're just trying to get moving again. you know, "I'm just here to help and support my teammates. I'm going to do bugs, the annoying things on somebody else's plate or that no one has been able to get to. do grunt work, organizational stuff, write-ups. just things to help people out."

and the reason you want to do this is you want to move away from an outcome-based mindset to momentum-based. like, you want to build a routine, because momentum is everything in this, and relationships are an even bigger thing in this. and by getting momentum going and fixing your relationships with other people, the thing you get is stamina. and that's when you realize the real lesson, which is that big projects are not done with big efforts. it's a big marathon, just slow, steady, incremental work. forget about a weekly basis, like a daily basis, an hourly basis, to the point that it's muscle memory.

you're in automatic mode. you wake up, you make yourself coffee, you get to work, and you just do the work. and then 30 days later, you look back and you see the immense volume of work that you've done just by doing a little bit every day. because the thing you're going for is reliability, fixing your relationships. you want to take the burden off your teammates and your manager, because then, once you build that trust, they will give you bigger work. that's the move here.

remember, you are in the reputation-building business, and software is actually downstream of that. godspeed and best of luck.

How not to forget what matters

HENRIK KARLSSON · 20 JUL 2026 · [SOURCE](#)



1.

Every few months I will read a tweet, or have a conversation, that makes me feel *this is important, I must remember this*. Often, these epiphanies are accompanied by a sense that *I actually know this already*, it had just somehow slipped my mind.

And for a few days, I do remember: my life shimmers with a new intensity, and I live the truth of what I grasped. But then, inevitably, the conveyor belt of things to pay attention to keeps churning, and my mind gets filled with small problems I need to solve, or new epiphanies or random noise, like news, and the shining fades from my eyes—I regress to being the same person as ever.

The Latin word for the tendency to lose track of what matters in the cacophony of things that attract our attention is *stultia*. “Stultia,” writes Michel Foucault in “[Self-writing](#),”

is defined by mental agitation, distraction, change of opinions and wishes, and consequently weakness in the face of all the events that may occur; it is also characterized by the fact that it turns the mind toward the future, makes it interested in novel ideas, and prevents it from providing a fixed point for itself in the possession of acquired truth.

You can't just read a blog post about high agency, get filled with a sense of possibility, and become, from then on, an agentic person. As John Gray puts it in his monograph on J.S. Mill, our character is "a cluster of habitual willings." For changes to our behavior to become permanent, we must become different people.

In the same way that it is not enough to make a *resolution* that you will learn the piano, it is not enough to *realize* that when the kids act out, you shouldn't lose your temper but slow down, listen, and regulate their nervous systems with the help of yours. Imagine how good a person I would be if having insights were enough! But reacting to the frustrations of your children with calm and curiosity is a skill as much as playing the piano is—and as with the piano, the act of learning it requires rewiring your nervous system through sustained attention and practice. Realizing the value of acting in a certain way might give you a temporary motivation to do it. But in order to actually live in accordance with what you believe in long-term, you must make it a habit.

And this is much harder than making a habit out of playing the piano. When you're trying to make something like piano practice a habit, the standard advice is to chain it onto some already existing habit—to practice immediately after you brush your teeth in the morning, for example, or after you change out of your work clothes in the afternoon. Chaining the new habit to an already existing one provides a predictable trigger that helps remind you to practice. But the habits that make up our characters often do not follow a predictable schedule like this. I never know, for instance, when our children will act out (except that it will usually be when I'm least capable of handling it with grace—whenever both they and I are unusually hungry and tired). The conflicts seem to come out of nowhere, so I have to, somehow, *always* be ready to act in the proper way. I need to have the right reaction "ready at hand" (*procheiron*), as the Greek-philosopher-Roman-slave Epictetus put it. If Johanna and I talked about how we want to deal with the kids' conflicts the night before, I will nearly always handle the situation well. The problem is to keep it top of mind.

2.

During the first two centuries of the Roman Empire, there spread a practice known as *hypomnēmata*, a type of notetaking system, used as a tool for meditation,^[1] in which the writer would store quotes from books they had read. Each day, often in the morning, the notetaker would open their notebook and look for a passage relevant to something they were struggling with, and then they would meditate on that—unpacking it, making the idea top of mind, ensuring it was alive in them. If they needed courage, for instance, they could meditate on an anecdote that made it real for them what it meant to act bravely. The idea was that over time, the insights they gathered by reading would be transformed into character, something deeply ingrained in their way of thinking and seeing and acting.

This was, as I understand it, an exercise designed to combat the problem I outlined above. Meditating on what matters is a simple habit, which you can chain onto your morning routine, but it reinforces the habits you can't plan, the habits that make up your character. It was, in the words of the French classicist Pierre Hadot, a spiritual exercise—an *exercise* because it required work and discipline, *spiritual* because it engaged the whole person, not just their intellect, but their emotions and their moral character. It was an attempt to treat the formation of character as a skilled practice, as something you can deliberately train and improve through targeted exercises.

The most famous example of this practice is the meditations Marcus Aurelius wrote in his tent as plague swept through the camps during the military campaigns along the Danube River, but it seems to have been a fairly widespread practice among the “cultivated class.” A suggestion repeated in several popular manuals for living was that you should collect every snippet of thought that deeply inspires you to live in a more ethically true way and then, in the pre-dawn hour, look through your hypomnēmata to find passages relevant to your current situation—insightful quotes, examples, actions you had witnessed, notes from conversations you'd had, and so on^[2]—and meditate, in writing, on those that help you orient toward your current challenges, until you feel inspired to act in the proper way.^[3]

What could be an example? Today, June 14, I woke up with a headache after having slept with my neck at an odd angle and so didn't feel like working. Having procrastinated for an hour or so, while Johanna tried to get me to start the day, I recalled principle 23 from [a list Nabeel S. Qureshi has compiled](#) for himself:

Doing things is energizing, wasting time is depressing. You don't need that much 'rest'.

Walking around the farm with a cup of coffee,^[4] reminding myself of the truth of this observation, I gained a sliver of motivation, enough to throw myself into rewriting this essay—and now my headache has lifted, and I feel excited. For more examples of things one might meditate on, I suggest looking at Nabeel's list. I also like to meditate on stories that make real to me some ethos I aspire to, such as the story of how Werner Herzog, dead broke while scouting for locations for *Nosferatu* in Brittany, happened upon a field of menhirs and decided to abandon his work to stay at the field for as long as necessary to solve the mystery of how the giant stones had been erected—a story that makes it visceral to me what it means to “take your curiosity seriously.”

The hypomnēma was a mechanism for centering your mind on what matters, and for gradually refining your understanding of what matters. It was, to use a word from Plutarch, a tool for *ethopoiesis* (*ethos* meaning “character” and *poiesis* “making”), a tool for turning truth into character. By meditating daily on sentences that made it real to you how you wanted to live, you

would remember to do the right thing—you would remember to practice it during the day (simply thinking about it is not going to change you). And gradually, you would become that sort of person. You wouldn't even need to remind yourself. Your principles would have become your character. You would have developed expertise in the skill of holding yourself in a better way.

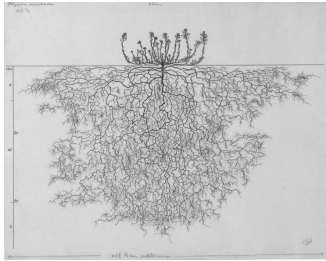
3.

I first came across the idea of using hypomnēmata in this way eight years ago when a friend sent me a copy of Foucault's "Self-writing." In the essay, Foucault talks about hypomnēmata and other modes of reading and writing used to fashion a self during the Hellenistic and Imperial eras. I didn't pick up the practice in its full form. But rereading the essay now, I realize how much it has influenced how I write: my essays are (often) meditations I do in order to deepen core ideas I want to live by, and to strengthen parts of myself I want strengthened.^[5] (Writing treatises and letters was a common way of internalizing the content of the hypomnēmata.) And this practice has been transformative for me. I have often noticed that my experience of reality improves if I write and think about something.

But it strikes me now that the practice Foucault wrote about was probably more transformative than what I've ended up doing. Essay writing is incredibly time-consuming, and a lot of that time is spent on things that aren't self-transforming: I spend less time reshaping my mind than I spend solving literary-technical problems that help me write more functional and beautiful essays, for the joy of the craft and for the benefit of readers. Another limitation of my practice is that when an essay is done, I move on. The ideas—though they have been much deepened and more firmly lodged in my mind—fall out of attention and start to fade.

There is an element of self-deception involved here. I like to write essays, so it is comforting to think of it as a powerful practice, something that helps me live more fully and grow as a person. But if I look at it soberly, it is clear to me that essay writing is not a practice that is ideal for the purpose of ethopoiesis.^[6] It is common to think that what we do achieves what we want it to achieve, even if there is no evidence for it. There are many practices that promise to transform and improve us—therapy, meditation, psychedelics—, but that branding doesn't mean that they actually do much for us: it is common to see people use these techniques for years without any obvious progress on their problems. If you want to achieve a particular outcome, it is important to start from that goal and evaluate which practices actually help you.

The most important ideas we need to return to weekly, even daily. Essay-writing, then, is not a functional substitute for having a practice that keeps the important truths top of mind, day after day. But it did help me reach that conclusion.



How to think in writing

Credits

This essay is based on a fragment from a longer essay on spiritual exercises that I wrote. That essay didn't work, and Johanna suggested using a few of the paragraphs for a new essay. She also suggested the main argument and general outline of the body of this essay, reorganized the second draft to shift the emphasis to what we came to see as the main idea, and corrected several faulty lines of argument I made.

*The ideas were influenced by reading Pierre Hadot's *Philosophy as a Way of Life* and Michel Foucault's *The Hermeneutics of the Subject*, *The Care of the Self*, and "[Self-Writing](#)." I was influenced by several conversations with Michael Nielsen and his [notes on Tanya Luhrmann's How God Becomes Real](#), as well as a conversation with Alexander Obenauer about his practice. Claude Fable 5 (RIP) corrected four factual errors.*

1 [find in text]

The practice of keeping hypomnēmata had already existed for more than 400 years at this point, but the practices around it shifted significantly around this era, as I understand it, from a more conventional type of notetaking into the more specialized type of meditation described in this essay.

2 [find in text]

How do you actually keep these notes? A lot of people have insights and write them into Apple Notes, but then they never look at them again, and it becomes a mess after a while, so when they need the idea, it is nowhere to be found. I'm not sure exactly how the Romans managed to find their way back to the right wax tablet or papyrus. I guess the easiest thing is to just keep a single document for the most important ideas and stories, with short memorable concept handles (länks) that help you remember each. If you can't be bothered making your notes structured, though, LLMs are powerful enough now that they can trawl through your messy notes and find the

relevant quotes. Just give Claude Code or Codex access to your notes and ask it to find segments relevant to whatever you are thinking about. This will not be as powerful as doing the work yourself, but it is better than just forgetting.

3 [find in text]

I read somewhere that when meditating on their hypomnēmata, the Romans and Hellenistic Greek would also set intentions for themselves, principles they want to live by that day. I can't find the source for this claim now, but it resonated with something Johanna often tells me: it is much easier to hold the line if you've articulated your position to yourself beforehand. If, for example, I'm on a call with someone who wants to propose a project, I will often get caught up in their excitement, or feel uncomfortable setting the right boundaries—I often don't even know what the right boundaries are, because there is too much to consider for me to figure out my position in real time—and so I will say yes to things I shouldn't. But if I have thought things through beforehand, and have articulated to myself what I'm trying to achieve, and what my principles are, I can more easily see if this is in line with what I want, and I can clearly articulate my position. I wish I did this more often. Or at least remembered the principle to always ask for time to think things through on my own before committing.

4 [find in text]

Seneca and Epictetus would have insisted on making meditating in writing. The argument is that writing engages you more deeply, transforming what you have read “into tissue and blood” (in vivres et in sanguinem). Using an oft-repeated metaphor, Seneca writes, “We should imitate bees, who wander and pluck from flowers suited for making honey, then arrange and distribute through the combs what they have brought in... We must digest what we have read; otherwise it passes into the memory, not into the mind.” Food, he says, is a burden so long as it stays what it was; only when it changes into blood and tissue does it become strength. It might feel like I understand a topic without writing about it, but if I earnestly try to unpack my thinking in writing, my ignorance is revealed, and my thinking transformed.

5 [find in text]

I'm not sure how much of the shift in my writing came directly from reading Foucault: it has been something that I've iterated myself toward. I've gradually written more with the intention of shaping my character and my attention, and this has been driven by the experience of deepened presence this has afforded me. But I'm pretty sure the ideas from that essay provided a template that helped me make sense and direct the evolution of my writing. I know I have often returned to the essay, and have used Plutarch's term ethopoetical writing to make sense to what I'm trying to do.

What it can do well, though, is to push deep into an idea and to restructure your thoughts about something. It is like an intervention where you want to shake things up properly, a three week session where you think and think and think about the same thing all the time. It is, I think, a useful complement to a daily practice.

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 24 APR 2026 · [SOURCE](#)

Get started with Claude Managed Agents by following our [docs](#).

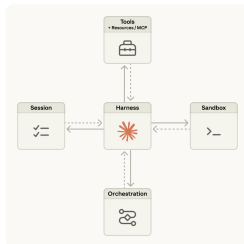
A running topic on the Engineering Blog is how to [build effective agents](#) and [design harnesses](#) for [long-running work](#). A common thread across this work is that harnesses encode assumptions about what Claude can't do on its own. However, those assumptions need to be frequently questioned because they can [go stale](#) as models improve.

As just one example, in prior work [we found](#) that Claude Sonnet 4.5 would wrap up tasks prematurely as it sensed its context limit approaching—a behavior sometimes called “context anxiety.” We addressed this by adding context resets to the harness. But when we used the same harness on Claude Opus 4.5, we found that the behavior was gone. The resets had become dead weight.

We expect harnesses to continue evolving. So we built Managed Agents: a hosted service in the Claude Platform that runs long-horizon agents on your behalf through a small set of interfaces meant to outlast any particular implementation—including the ones we run today.

Building Managed Agents meant solving an old problem in computing: how to design a system for “[programs as yet unthought of](#).” Decades ago, operating systems solved this problem by virtualizing hardware into abstractions—*process*, *file*—general enough for programs that didn't exist yet. The abstractions outlasted the hardware. The `read()` command is agnostic as to whether it's accessing a disk pack from the 1970s or a modern SSD. The abstractions on top stayed stable while the implementations underneath changed freely.

Managed Agents follow the same pattern. We virtualized the components of an agent: a session (the append-only log of everything that happened), a harness (the loop that calls Claude and routes Claude's tool calls to the relevant infrastructure), and a sandbox (an execution environment where Claude can run code and edit files). This allows the implementation of each to be swapped without disturbing the others. We're opinionated about the shape of these interfaces, not about what runs behind them.



We started by placing all agent components into a single container, which meant the session, agent harness, and sandbox all shared an environment. There were benefits to this approach, including that file edits are direct syscalls, and there were no service boundaries to design.

But by coupling everything into one container, we ran into an old infrastructure problem: we'd adopted a *pet*. In the pets-vs-cattle analogy, a pet is a named, hand-tended individual you can't afford to lose, while cattle are interchangeable. In our case, the server became that pet; if a container failed, the session was lost. If a container was unresponsive, we had to nurse it back to health.

Nursing containers meant debugging unresponsive stuck sessions. Our only window in was the WebSocket event stream, but that couldn't tell us *where* failures arose, which meant that a bug in the harness, a packet drop in the event stream, or a container going offline all presented the same. To figure out what went wrong, an engineer had to open a shell inside the container, but because that container often also held user data, that approach essentially meant we lacked the ability to debug.

A second issue was that the harness assumed that whatever Claude worked on lived in the container with it. When customers asked us to connect Claude to their virtual private cloud, they had to either peer their network with ours, or run our harness in their own environment. An assumption baked into the harness became a problem when we wanted to connect it to different infrastructure.

The solution we arrived at was to decouple what we thought of as the “brain” (Claude and its harness) from both the “hands” (sandboxes and tools that perform actions) and the “session” (the log of session events). Each became an interface that made few assumptions about the others, and each could fail or be replaced independently.

The harness leaves the container. Decoupling the brain from the hands meant the harness no longer lived inside the container. It called the container the way it called any other tool: `execute(name, input) → string`. The container became cattle. If the container died, the harness caught the failure as a tool-call error and passed it back to Claude. If Claude decided to retry, a new container could be reinitialized with a standard recipe: `provision({resources})`. We no longer had to nurse failed containers back to health.

Recovering from harness failure. The harness also became cattle. Because the session log sits outside the harness, nothing in the harness needs to survive a crash. When one fails, a new one can be rebooted with `wake(sessionId)`, use `getSession(id)` to get back the event log, and resume from the last event. During the agent loop, the harness writes to the session with `emitEvent(id, event)` in order to keep a durable record of events.

Component	Interface (Contracts)	Satisfied by
Session	<code>getSession(session_id) → {Session, Event[]}</code> <code>emitEvent(session_id, event) → void</code> <code>getSessionEvents(session_id) → Event[]</code> <code>provision({resources}) → Container</code>	Any append-only log that can be constructed in order from any event pattern and supports dependency resolution— Postgres, Redis, in-memory array, etc.
Orchestration	<code>wake(session_id) → void</code>	Any scheduler that can call a function with an ID and any on failure— a cron job, a process manager, a container orchestrator, etc.
Harness	<code>void Effects[] → Effects[]</code> <code>void Effects[] → void</code>	Any loop that yields effects and appends progress to the Session.
Sandbox	<code>provision({resources}) → Container</code> <code>execute(container, tool) → string</code> <code>kill(container) → void</code>	Any executor that can be configured and executed any number of times— a cloud process, a remote container, etc.
Resources	<code>{resource_ref, secret, url[]}</code>	Any durable store the container can look back for information— Postgres, Redis, a git remote, etc.
Tools	<code>name, description, input_schema</code>	Any capability observable as a name and/or input schema— MCP servers, custom tools, etc.

The security boundary. In the coupled design, any untrusted code that Claude generated was run in the same container as credentials—so a prompt injection only had to convince Claude to read its own environment. Once an attacker has those tokens, they can spawn fresh, unrestricted sessions and delegate work to them. Narrow scoping is an obvious mitigation, but this encodes an assumption about what Claude can't do with a limited token—and Claude is getting increasingly smart. The structural fix was to make sure the tokens are never reachable from the sandbox where Claude’s generated code runs.

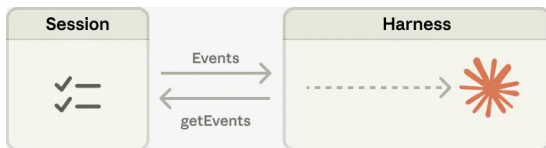
We used two patterns to ensure this. Auth can be bundled with a resource or held in a vault outside the sandbox. For Git, we use each repository’s access token to clone the repo during sandbox initialization and wire it into the local git remote. Git `push` and `pull` work from inside the sandbox without the agent ever handling the token itself. For custom tools, we support MCP and store OAuth tokens in a secure vault. Claude calls MCP tools via a dedicated proxy;

this proxy takes in a token associated with the session. The proxy can then fetch the corresponding credentials from the vault and make the call to the external service. The harness is never made aware of any credentials.

The session is not Claude’s context window

Long-horizon tasks often exceed the length of Claude’s context window, and the standard ways to address this all involve irreversible decisions about what to keep. We’ve explored these techniques in [prior work](#) on context engineering. For example, compaction lets Claude save a summary of its context window and the memory tool lets Claude write context to files, enabling learning across sessions. This can be paired with context trimming, which selectively removes tokens such as old tool results or thinking blocks.

But irreversible decisions to selectively retain or discard context can lead to failures. It is difficult to know which tokens the future turns will need. If messages are transformed by a compaction step, the harness removes compacted messages from Claude’s context window, and these are recoverable only if they are stored. Prior work [has explored](#) ways to address this by storing context as an object that lives *outside* the context window. For example, context can be an object in a REPL that the LLM programmatically accesses by writing code to filter or slice it.



In Managed Agents, the session provides this same benefit, serving as a context object that lives outside Claude’s context window. But rather than be stored within the sandbox or REPL, context is durably stored in the session log. The interface, `getEvents()`, allows the brain to interrogate context by selecting positional slices of the event stream. The interface can be used flexibly, allowing the brain to pick up from wherever it last stopped reading, rewinding a few events before a specific moment to see the lead up, or rereading context before a specific action.

Any fetched events can also be transformed in the harness before being passed to Claude’s context window. These transformations can be whatever the harness encodes, including context organization to achieve a high prompt cache hit rate and context engineering. We separated the concerns of recoverable context storage in the session and arbitrary context management in the harness because we can’t predict what specific context engineering will be required in future models. The interfaces push that context management into the harness, and only guarantee that the session is durable and available for interrogation.

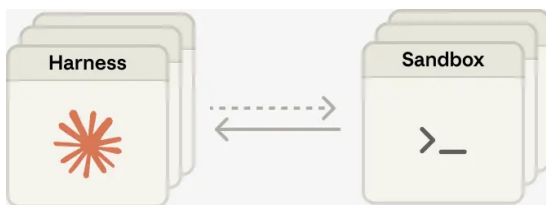
Many brains. Decoupling the brain from the hands solved one of our earliest customer complaints. When teams wanted Claude to work against resources in their own VPC, the only path was to peer their network with ours, because the container holding the harness assumed every resource sat next to it. Once the harness was no longer in the container, that assumption went away. The same change had a performance payoff. When we initially put the brain in a container, it meant that many brains required as many containers. For each brain, no inference could happen until that container was provisioned; every session paid the full container setup cost up front. Every session, even ones that would never touch the sandbox, had to clone the repo, boot the process, fetch pending events from our servers.

That dead time is expressed in time-to-first-token (TTFT), which measures how long a session waits between accepting work and producing its first response token. TTFT is the latency the user most acutely *feels*.

Decoupling the brain from the hands means that containers are provisioned by the brain via a tool call (`execute(name, input) → string`) only if they are needed. So a session that didn't need a container right away didn't wait for one. Inference could start as soon as the orchestration layer pulled pending events from the session log. Using this architecture, our p50 TTFT dropped roughly 60% and p95 dropped over 90%. Scaling to many brains just meant starting many stateless harnesses, and connecting them to hands only if needed.

Many hands. We also wanted the ability to connect each brain to many hands. In practice, this means Claude must reason about many execution environments and decide where to send work—a harder cognitive task than operating in a single shell. We started with the brain in a single container because earlier models weren't capable of this. As intelligence scaled, the single container became the limitation instead: when that container failed, we lost state for every hand that the brain was reaching into.

Decoupling the brain from the hands makes each hand a tool, `execute(name, input) → string`: a name and input go in, and a string is returned. That interface supports any custom tool, any MCP server, and our own tools. The harness doesn't know whether the sandbox is a container, a phone, or a Pokémon emulator. And because no hand is coupled to any brain, brains can pass hands to one another.



The challenge we faced is an old one: how to design a system for “programs as yet unthought of.” Operating systems have lasted decades by virtualizing the hardware into abstractions general enough for programs that didn't exist yet. With Managed Agents, we aimed to design a system that accommodates future harnesses, sandboxes, or other components around Claude.

Managed Agents is a meta-harness in the same spirit, unopinionated about the *specific* harness that Claude will need in the future. Rather, it is a system with general interfaces that allow many different harnesses. For example, Claude Code is an excellent harness that we use widely across tasks. We’ve also shown that task-specific agent harnesses excel in narrow domains. Managed Agents can accommodate any of these, matching Claude’s intelligence over time.

Meta-harness design means being opinionated about the interfaces around Claude: we expect that Claude will need the ability to manipulate state (the session) and perform computation (the sandbox). We also expect that Claude will require the ability to scale to many brains and many hands. We designed the interfaces so that these can be run reliably and securely over long time horizons. But we make no assumptions about the number or location of brains or hands that Claude will need.

Written by Lance Martin, Gabe Cemaj and Michael Cohen. Special thanks to the Agents API team and Jake Eaton for their contributions.